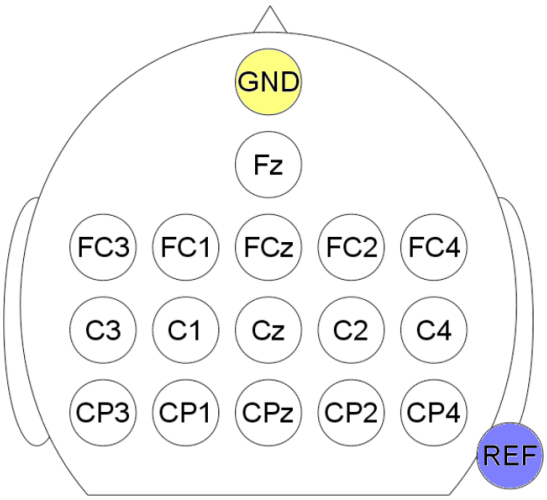
Coding Exercise: Analysis of Event-related Potentials of error recognition

Goals

This exercise is designed to:

- Provide an example of event-related potential (ERP) BCI analysis in the time domain.
- Provide an overview of signal processing (spatial filtering, spectral filtering, etc) and machine learning (feature selection, dimensionality reduction, classification) methods used in BCI.

Description



You are given a dataset of a single subject consisting of 6 EEG files stored in GDF format. Each file corresponds to a single run/block of a protocol allowing the subject to input text with a motor imagery (MI) BCI and correct potential errors through error-potential (ErrP) detection. Each file corresponds to a single written word (name of an animal). Signals were collected at a sampling rate of 512 Hz. The EEG data were recorded at 16 scalp locations according to the 10-20 system (see picture on the right, channel indices start at Fz and proceed rowise, e.g., the index of C3 is 7 and of CP4 16). Data can be loaded with the biosig toolbox as: [data, header] = sload('FILEPATH'); . All events are stored in the header of the loaded file, in structure header.EVENT: TYP for the type of event, POS for the position (pointer to the data matrix' time indices) and DUR for the duration of an event (not used). In this protocol, a binary speller was evaluated, where users could move a cursor left/right with a 2-class MI BCI, to get the cursor closer to the desired letter (i.e., target letter to be typed), until the latter was typed. In case of an erroneous command, the latest command is reversed ("undo"). The experiment (exact speller interface and control paradigm, MI and ErrP analysis methods) is described elaborately in the open access paper "On Error-Related Potentials During Sensorimotor-Based Brain-Computer Interface: Explorations With a Pseudo-Online Brain-Controlled Speller" which can be downloaded from ieeexplore.ieee.org. The supplementary material can also be downloaded under "Media". In this exercise you will focus on the ErrP analysis. We assume that after each erroneous MI command forwarded to the speller through the

MI BCI an ErrP waveform is elicited. The neurophysiological data can be downloaded from this link: Zenodo link

Tasks

1. Extract trials of "Error" and "Correct" brain responses to the speller cursor's movements. We assume that the onset (t=0) of ErrPs must coincide with the movement of the cursor. The latter is marked as an 897 event for successful commands and 898 for erroneous ones. One second after an 897/898 event the user received feedback confirming the presence or not of an error committed (red or green square, respectively). These events are marked as 3 (correct) and 4 (error). From the literature, we know that ErrP waveforms last approximately 1 second. You can perform the trial extraction accordingly. Your code should iterate over all the available files to make available all available trials for this subject. Reflect on and discuss any implications of saving events as pulses in a synced trigger/status channel (hardware trigger) or in the file header (software trigger).
2. Compute the percentage of Error and Correct trials. Are they balanced? If not, which class is more frequent? How does this relate to the "oddball paradigm" concept that is relevant to ErrPs? How was this achieved experimentally in this protocol? What are the implications on the usage of ErrPs for corrections in realistic BCI control scenarios?
3. Treat all trials with a band-pass Butterwirth filter with cut-off at [1, 10] Hz and baseline each channel of each trial with the value at t=0 (i.e., for each channel, remove form all time samples the potential at time t=0). To set up and apply a band-pass filter, check the use and documentation of MATLAB commands **butter** and **filtfilt**. What are the implications of performing filtering on the whole file instead of the 1-sec segment corresponding to a single trial? Why should we prefer in this case baseline removal with the value at t=0 instead of DC removal?
4. Calculate and plot grand average Error and Correct waveforms and their difference Error-Correct on all 16 channels. Inspect the grand averages and compare them to the figures you have seen in the book and the respective references. Is there good correspondence between the two? Which time points show the greatest ERP difference between Error and Correct trials?
5. In the analysis described in the paper, trials were aligned in time. Compare the figures in the paper with those you have created here. Discuss the usefulness of this re-alignment and the implications it could have for closed-loop (online), real-time ErrP detection.
6. For the salient timepoints (just select them visually from the grand average plots), plot the scalp distribution of the Difference of potential between Error and Correct. Discuss whether this is what you would expect according to the discussion in class.
7. Inspecting the topoplots and grand averages, infer whether individual channel ErrP waveforms differ substantially. Based on this observation, can you predict the effects of CAR/Laplacian spatial

filtering in this scenario? Add CAR and/or Laplacian spatial filtering and recompute and plot the grand averages to verify your intuition.

8. Clearly, classifying responses to the classes "Correct" and "Error" must be related to the waveforms associated to the two conditions on the recorded EEG channels. Discuss what are the natural candidate features in this problem, to be fed into a pattern recognition/machine learning classifier. Which dimensions of the problem are involved? How many candidate features do we have? How does this number compare to the number of available trials?

9. In our attempt to reduce the number of features, one could imagine that the shape of the Correct and Error ErrP waveforms may be adequately captured with lower time sampling. Using MATLAB function **downsample**, come up with a good downsampling ratio that preserves the essential ErrP information. Compare the new number of candidate features with the number of available trials. Is there a further need for reducing the number of attributes/features to be used for classification? How can we achieve this?

10. Using the given resources fisherscore.m and rsquared.m apply the corresponding "filter methods" for feature selection (i.e., rank the features according to their Fisher Score or r^2 fitness value), either on the original and/or on the downsampled data. Plot the candidate features' fitness in both cases as heatmaps (use MATLAB's **imagesc**). Are the "good" features in the anticipated time points and channels (given the grand average and topographical plots produced above)? Do the two methods differ substantially? Discuss the pros and cons of Fisher Score and r^2. Reflect on other statistical methods that can be used for feature selection.

11. Using MATLAB's **pca** command perform Principal Component Analysis. Discuss what each of the outputs of **pca** represents and explain the dimensions of the different variables.

12. Using MATLAB's machine learning toolbox, train various classification models (e.g., LDA, SVM, etc.) and test their classification accuracy with Leave-One-Out (LOO) cross validation (CV). In LOO-CV, we train a model on all available samples but a single one that we use for testing. We repeat this procedure for as many samples as there are available in the dataset. If we have N samples (trials in this case) available, each trial will have been used once for testing and N-1 times for training. Keep the classifier's testing predictions for each sample/trial in an array. Use this array to produce a classification accuracy estimate. Before training/testing, in the LOO-CV iterations, select the number of N-best features you want to use according to the feature selection ranking. MATLAB provides a great many options for machine learning models to use. Visit this link: https://uk.mathworks.com/help/stats/classification.html and select one or more models to compare. All models can be trained (i.e., learn/estimate the parameters from data and labels, what we called supervised learning) with the command Model = **fit*modelname***(traindata, trainlabels); . The modelame varies according to the method used (check link above). The prediction for the testing samples can be derived with Prediction = predict(Model, testdata); The testdata may be

composed of a single or multiple samples/trials. Check the effect of the number of features used on classification accuracy. Compare different classification methods.

13. Repeat the previous step, but instead of reducing the dimensionality of your problem with feature selection, use PCA dimensionality reduction. Keeping the same number of "features", compare the effects on classification accuracy. Pay attention to the fact that MATLAB's pca will by default center the data (remove the mean vector—mean of all samples in the dataset--from all data samples) before computing the principal components (eigenvectors of the covariance matrix) and the loadings (eigenvalues). Hence, before applying PCA to the testing trial, you need to center it, too.
14. Is the classification accuracy achieved satisfactory? Compute the accuracy separately for each class and/or compute the "confusion matrix" of the classification problem (CxC matrix, where C the number of classes, where element C_{ij} contains the number of samples that belong to class i—the ground truth label--and were predicted by the classifier as belonging to class j). Is the per-class accuracy balanced? Why is that the case? Is any of the tried models superior form the others in that respect? Discuss how this problem could be overcome. Check the information on the book about the concept of regularization.